

“The Essentials of Computer Organization and Architecture”

Unit 4 – Exercises and Solutions

LEBE1 A computer uses 32 bit integers and has a byte-addressable main memory. Show how the value 1234_{16} will be stored on that memory, starting at address 0_2 , if:

- a) The computer is a **Big Endian** machine.
- b) The computer is a **Little Endian** machine.

LEBE1 A computer uses 32 bit integers and has a byte-addressable main memory. Show how the value 1234_{16} will be stored on that memory, starting at address 0_2 , if:

- a) The computer is a Big Endian machine.**
- b) The computer is a Little Endian machine.**

Solution:

Address 	00	01	10	11
Big Endian	00	00	12	34
Little Endian	34	12	00	00

LEBE2 A computer uses 32 bit integers and has a byte-addressable main memory. Show how the values below are stored on that memory, starting at address 10_{16} , for the little endian and big endian alternatives.

a) **$456789A1_{16}$**

b) **$0000058A_{16}$**

c) **14148888_{16}**

LEBE2 A computer uses 32 bit integers and has a byte-addressable main memory. Show how the values bellow are stored on that memory, starting at address 10_{16} , for the little endian and big endian alternatives.

a) $456789A1_{16}$

b) $0000058A_{16}$

c) 14148888_{16}

Solution: a)

Address $\longrightarrow$	10_{16}	11_{16}	12_{16}	13_{16}
Big Endian	45	67	89	A1
Little Endian	A1	89	67	45

b)

Address $\longrightarrow$	10_{16}	11_{16}	12_{16}	13_{16}
Big Endian	00	00	05	8A
Little Endian	8A	05	00	00

c)

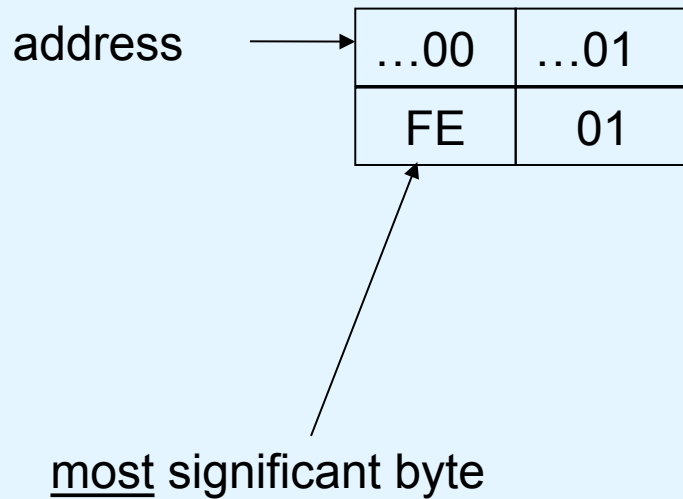
Address $\longrightarrow$	10_{16}	11_{16}	12_{16}	13_{16}
Big Endian	14	14	88	88
Little Endian	88	88	14	14

LEBE3 The first 2 bytes of a 2M x 16 main memory hold the values FE_{16} and 01_{16} . Together, these bytes represent a 16-bit two's complement integer number. What is the corresponding decimal value of the number:

- a) If the number was stored in the Big Endian format.**
- b) If the number was stored in the Little Endian format.**

LEBE3 The first 2 bytes of a 2M x 16 main memory hold the values FE_{16} and 01_{16} . Together, these bytes represent a 16-bit two's complement integer number. What is the corresponding decimal value of the number:

a) Big Endian – Solution:



value is **negative**

$$FE01_{16} = 1111\ 1110\ 0000\ 0001_2$$

discovering the modulus of the number:

one's complement:

$$1111\ 1110\ 0000\ 0001_2 \rightarrow$$

$$0000\ 0001\ 1111\ 1110_2$$

one unit sum:

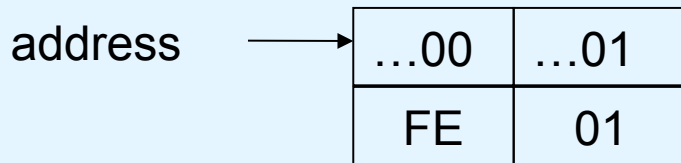
$$0000\ 0001\ 1111\ 1110_2 + 1_2 =$$

$$0000\ 0001\ 1111\ 1111_2 = 511_{10}$$

conclusion: the stored value is -511_{10}

LEBE3 The first 2 bytes of a 2M x 16 main memory hold the values FE_{16} and 01_{16} . Together, these bytes represent a 16-bit two's complement integer number. What is the corresponding decimal value of the number:

b) Little Endian – Solution:



value is **positive**

$$01FE_{16} = \textcolor{red}{0}000\ 0001\ 1111\ 1110_2 = 510_{10}$$

least significant byte

conclusion: the stored value is 510_{10}

LEBE4 On a computer with a Little-Endian architecture and 16-bit integers (represented in sign and magnitude), the first 4 bytes of RAM have the following value (in hexadecimal): 1234AABB.

- a)** How many 16-bit integers are stored in the first 4 bytes of the RAM ?
- b)** Extract from memory and reconstruct the integer numbers identified in a). Present these numbers, in hexadecimal, as humans usually represent numbers (in Big-Endian).
- c)** Present, in decimal, the numbers identified in b).
- d)** Add the two decimal numbers presented in c) and present the value of the sum, in hexadecimal, in the way that humans commonly represent numbers (in Big-Endian). Note: in case the sum produces overflow/underflow, preserve the sign (that is, do not use the sign bit to accommodate the overflow/underflow).
- e)** Show how the value of the sum presented in d) would be stored in the RAM memory of this computer, assuming that the storage starts at byte 100.

LEBE4 On a computer with a Little-Endian architecture and 16-bit integers (represented in sign and magnitude), the first 4 bytes of RAM have the following value (in hexadecimal): 1234AABB.

a) How many 16-bit integers are stored in the first 4 bytes of the RAM ?

Solution:

2

Justification: if the integers are 16 bits, this corresponds to 2 bytes; hence, in the first 4 bytes of memory there will be 2 numbers (one number for every 2 bytes).

LEBE4 On a computer with a Little-Endian architecture and 16-bit integers (represented in sign and magnitude), the first 4 bytes of RAM have the following value (in hexadecimal): 1234AABB.

b) Extract from memory and reconstruct the integer numbers identified in a). Present these numbers, in hexadecimal, as humans usually represent numbers (in Big-Endian)..

Solution:

3412

BBAA

Justification: in memory are then the numbers 1234 (16) and AABB (16); since they are stored in Little-Endian, these numbers were originally 3412 (16) and BBAA (16); note that each number occupies 2 bytes, and each byte needs 2 hexadecimal digits.

LEBE4 On a computer with a Little-Endian architecture and 16-bit integers (represented in sign and magnitude), the first 4 bytes of RAM have the following value (in hexadecimal): 1234AABB.

c) Present, in decimal, the numbers identified in b).

Solution:

13330

-15274

Justification: we have already seen that the 1st number in memory is 3412 (16) and the 2nd is BBAA (16); considering they are in sign and magnitude, their value in base 10 is:

- $3412_{(16)} = \mathbf{0}011\ 0100\ 0001\ 0010_{(2)} = \mathbf{+}\ 011\ 0100\ 0001\ 0010_{(2)} = 13330_{(10)}$
- $BBAA_{(16)} = \mathbf{1}011\ 1011\ 1010\ 1010_{(2)} = \mathbf{-}\ 011\ 1011\ 1010\ 1010_{(2)} = -15274_{(10)}$

LEBE4 On a computer with a Little-Endian architecture and 16-bit integers (represented in sign and magnitude), the first 4 bytes of RAM have the following value (in hexadecimal): 1234AABB.

d) Add the two numbers presented in c) and present the value of the sum, in hexadecimal, in the way that humans commonly represent numbers (in Big-Endian). Note: in case the sum produces overflow/underflow, preserve the sign (that is, do not use the sign bit to accommodate the overflow/underflow).

Solution:

8798

Justification: the numbers identified in c) are 13330 (10) and -15274 (10) and its sum is -1944 (10); in binary (and using only 15 bits, once the 16th will be for the **sign**), the absolute value of the number corresponds to 000 0111 1001 1000 (2); adding the **sign bit**, which is **1**, once the number is negative, the value of the sum, in sign and magnitude with 16 bits is -1944 (10) = **1**000 0111 1001 1000 (2) = **8798** (16).

LEBE4 On a computer with a Little-Endian architecture and 16-bit integers (represented in sign and magnitude), the first 4 bytes of RAM have the following value (in hexadecimal): 1234AABB.

e) Show how the value of the sum presented in d) would be stored in the RAM memory of this computer, assuming that the storage starts at byte 100.

Solution:

Byte 100: 98

Byte 101: 87

Justification: the number calculated in d) is 8798 (16) and this representation corresponds to the one that humans usually use (Big Endian); as the number has to be stored in memory in Little Endian, first it is stored 98 (least significant part) and only then it is stored 87 (most significant part).

LEBE5 On a computer with a Big-Endian architecture and 16-bit integers (represented in two's complement), the first 4 bytes of RAM have the following value (in hexadecimal): 98000089.

- a)** How many 16-bit integers are stored in the first 4 bytes of the RAM ?
- b)** Extract from memory and reconstruct the integer numbers identified in a). Present these numbers, in hexadecimal, as humans usually represent numbers (in Big-Endian).
- c)** Present, in decimal, the numbers identified in b).
- d)** Add the two decimal numbers presented in c) and present the value of the sum, in hexadecimal, in the way that humans commonly represent numbers (in Big-Endian). Note: in case the sum produces overflow/underflow, preserve the sign (that is, do not use the sign bit to accommodate the overflow/underflow).
- e)** Show how the value of the sum presented in d) would be stored in the RAM memory of this computer, assuming that the storage starts at byte 10F.

LEBE5 On a computer with a Big-Endian architecture and 16-bit integers (represented in two's complement), the first 4 bytes of RAM have the following value (in hexadecimal): 98000089.

a) How many 16-bit integers are stored in the first 4 bytes of the RAM ?

Solution:

2

Justification: if the integers are 16 bits, this corresponds to 2 bytes; hence, in the first 4 bytes of memory there will be 2 numbers (one number for every 2 bytes).

LEBE5 On a computer with a Big-Endian architecture and 16-bit integers (represented in two's complement), the first 4 bytes of RAM have the following value (in hexadecimal): 98000089.

b) Extract from memory and reconstruct the integer numbers identified in a). Present these numbers, in hexadecimal, as humans usually represent numbers (in Big-Endian).

Solution:

9800
0089

Justification: in memory are then the numbers 9800 (16) and 0089 (16); since they are stored in Big-Endian, these numbers were originally 9800 (16) and 0089 (16); note that each number occupies 2 bytes, and each byte needs 2 hexadecimal digits.

LEBE5 On a computer with a Big-Endian architecture and 16-bit integers (represented in two's complement), the first 4 bytes of RAM have the following value (in hexadecimal): 98000089.

c) Present, in decimal, the numbers identified in b).

Solution:

-26624

137

Justification: we already saw that the 1st number in memory is 9800 (16) and the 2nd is 0089 (16); considering that they are in two's complement, the value in base 10 is:

- $9800_{(16)} = 1001\ 1000\ 0000\ 0000_{(2)}$; being a negative number in two's complement, the corresponding positive is obtained by inverting the bits and adding 1, that is,
 $0110\ 0111\ 1111\ 1111 + 1 = 0110\ 1000\ 0000\ 0000_{(2)} = +26624_{(10)}$, thus $9800_{(16)} = -26624_{(10)}$
- $0089_{(16)} = 0000\ 0000\ 1000\ 1001_{(2)}$; being a positive number in two's complement, its value is read directly from the bit sequence and so $0089_{(16)} = 137_{(10)}$

LEBE5 On a computer with a Big-Endian architecture and 16-bit integers (represented in two's complement), the first 4 bytes of RAM have the following value (in hexadecimal): 98000089.

d) Add the two decimal numbers presented in c) and present the value of the sum, in hexadecimal, in the way that humans commonly represent numbers (in Big-Endian). Note: in case the sum produces overflow/underflow, preserve the sign (that is, do not use the sign bit to accommodate the overflow/underflow).

Solution:

9889

Justification: the numbers identified in c) are -26624 (10) and 137 (10) and their sum is -26487 (10); being this a negative number, to know how it is represented in two's complement, its absolute value is represented in binary (using only 15 bits, since the 16th will be for the sign), its bits are inverted and 1 is added: $26487(10) = 110\ 0111\ 0111\ 0111\ (2)$; inverting it gives $001\ 1000\ 1000\ 1000\ (2)$ and adding 1 gives $001\ 1000\ 1000\ 1001\ (2)$; adding the sign bit, which is 1 (since the number is negative), we get $-26487\ (10) = 1001\ 1000\ 1000\ 1001\ (2) = \mathbf{9889}\ (16)$.

LEBE5 On a computer with a Big-Endian architecture and 16-bit integers (represented in two's complement), the first 4 bytes of RAM have the following value (in hexadecimal): 98000089.

e) Show how the value of the sum presented in d) would be stored in the RAM memory of this computer, assuming that the storage starts at byte 10F.

Solution:

Byte 10F: 98

Byte 110: 89

Justification: the number calculated in d) is 9889 (16) and this representation corresponds to the one that humans usually use (Big Endian); as the number has to be stored in memory also in Big Endian, first it is stored 98 (most significant part) and only then it is stored 89 (least significant part).

LEBE6 What problems may emerge in the context of a network data transfer between machines with different endianness, if precautions are not taken ? Exemplify with data from exercise LEBE3.

LEBE6 What problems may emerge in the context of a network data transfer between machines with different endianness, if precautions are not taken ? Exemplify with data from exercise LEBE3.

Problem:

The values will be misinterpreted if not properly converted before sending them or after receiving them. For the values of exercise LEBE3, this would translate in the following:

Sender	Receiver
Big	Little
-511	510
510	-511

Sender	Receiver
Little	Big
510	-511
-511	510

Solution:

- network is assumed to be Big-Endian (BE), and hosts BE or Little-Endian (LE)
- BE hosts do not need to convert data before its sending (direct copy of data from memory to network), nor after reception (direct copy of data from network to memory)
- LE hosts must convert data to BE before its sending (before copying to network), and from BE to LE after reception (before/when storing data in/to memory)

LEBE7 Write a C program that allows to check if the machine where the program is running follows the Big Endian or the Little Endian convention.

LEBE7 Write a C program that allows to check if the machine where the program is running follows the Big Endian or the Little Endian convention.

Solution:

```
#include <stdio.h>
int main()
{
    int a = 0x80000000; //32bit
    char *b = (char *) &a;
    if(b[0] != 0)
        printf("Big-Endian\n");
    else
        printf("Little-Endian\n");
}
```


SBA1 Convert the following infix expressions to the corresponding postfix version.

a. $X * Y + W * Z + V * U$

b. $W * X + W * (U * V + Z)$

c. $(W * (X + Y * (U * V))) / (U * (X + Y))$

Note:

- you may use the computationally friendly algorithm presented in the next slide (Homework: apply the algorithm at least to expression a.) ...
- ... or, alternatively, apply the heuristic taught in the classes.

INFIX to POSTFIX Conversion Algorithm

Notation: Q = *infix* expression; P = *postfix* expression derived from Q

1. Add the symbol “)” to Q and add (PUSH) the symbol “(“ to an empty stack
2. **For** each symbol from Q (read left to right), **Repeat** steps 3 to 6, **Until** the stack becomes empty
3. **If** the next symbol is “(“, **Then** add (PUSH) it to the stack
4. **If** the next symbol is an operand, **Then** add it to P
5. **If** the next symbol is an operator, **Then**
 - a. remove (POP) from the stack and add to P each operator with equal or bigger precedence than the operator found in Q
 - b. add (PUSH) to the stack the operator found in Q
6. **If** the next symbol is “)“, **Then**
 - a. remove (POP) from the stack and add to P each operator until a symbol “(“ is found at the top of the stack
 - b. remove (POP) the symbol “(“ from the stack and do not add it to P

SBA1 Convert the following infix expressions to the corresponding postfix version.

a. $X * Y + W * Z + V * U$

b. $W * X + W * (U * V + Z)$

c. $(W * (X + Y * (U * V))) / (U * (X + Y))$

Solutions:

a. $X Y * W Z * + V U * +$

b. $W X * W U V * Z + * +$

c. $W X Y U V * * + * U X Y + * /$

SBA2 Convert the following postfix expressions to the corresponding infix version.

a. W X Y Z - + *

b. U V W X Y Z + * + * +

c. X Y Z + V W - * Z + +

Hint: use the algorithm presented in the next slide , suitable to a computational implementation; basically, it is the algorithm used to evaluate an infix expression in a stack-based architecture; at the end of the process, the expression stored on the stack corresponds to the postfix version of the original infix expression

POSTFIX to INFIX Conversion Algorithm

Notation: P = *postfix* expression ; Q = *infix* expression derived from P

1. create an empty stack
2. **For** each symbol from P (read left to right), **Repeat** steps 3 to 4, **Until** the stack contains a single expression (which will be expression Q)
3. **If** the next symbol is an operand, **Then** add (PUSH) it to the stack
4. **If** the next symbol is an operator, **Then**
 - a. remove (POP) two expressions from the top of the stack
 - b. apply the operator to the expressions removed from the stack in a.
(the topmost will be the right side, and the one below will be the left side)
 - c. add (PUSH) to the stack the resulting expression from b.

SBA2 Convert the following postfix expressions to the corresponding infix version.

a. W X Y Z - + *

b. U V W X Y Z + * + * +

c. X Y Z + V W - * Z + +

Solutions:

a. $W * (X + (Y - Z))$

b. $U + V * (W + X * (Y + Z))$

c. $X + ((Y + Z) * (V - W) + Z)$

SBA3 a) Convert the following infix expression to postfix.

$$X = \frac{A - B + C * (D * E - F)}{G + H * K}$$

SBA3 a) Convert the following infix expression to postfix.

$$X = \frac{A - B + C * (D * E - F)}{G + H * K}$$

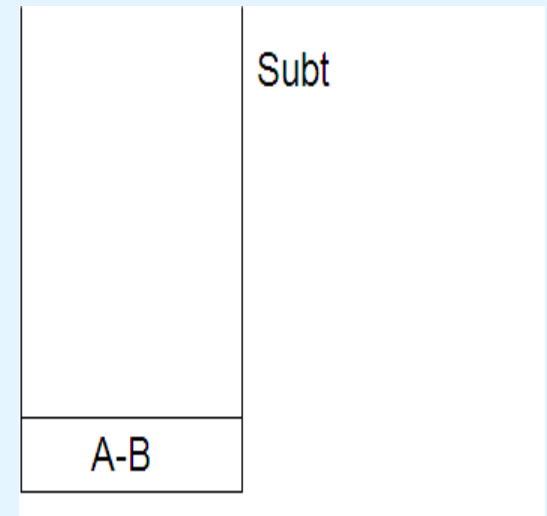
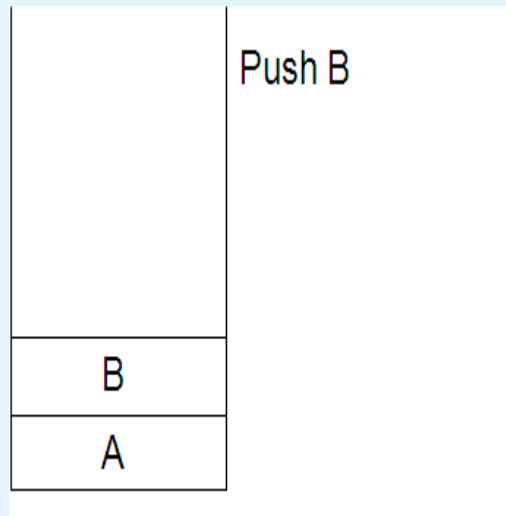
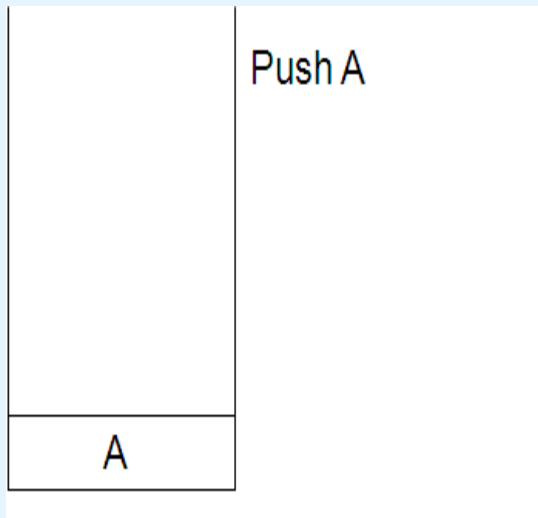
Solution – a)

A B - C D E * F - * + G H K * + /

SBA3 b) Write an assembly program that evaluates X using a stack-based architecture.

Solution – b)

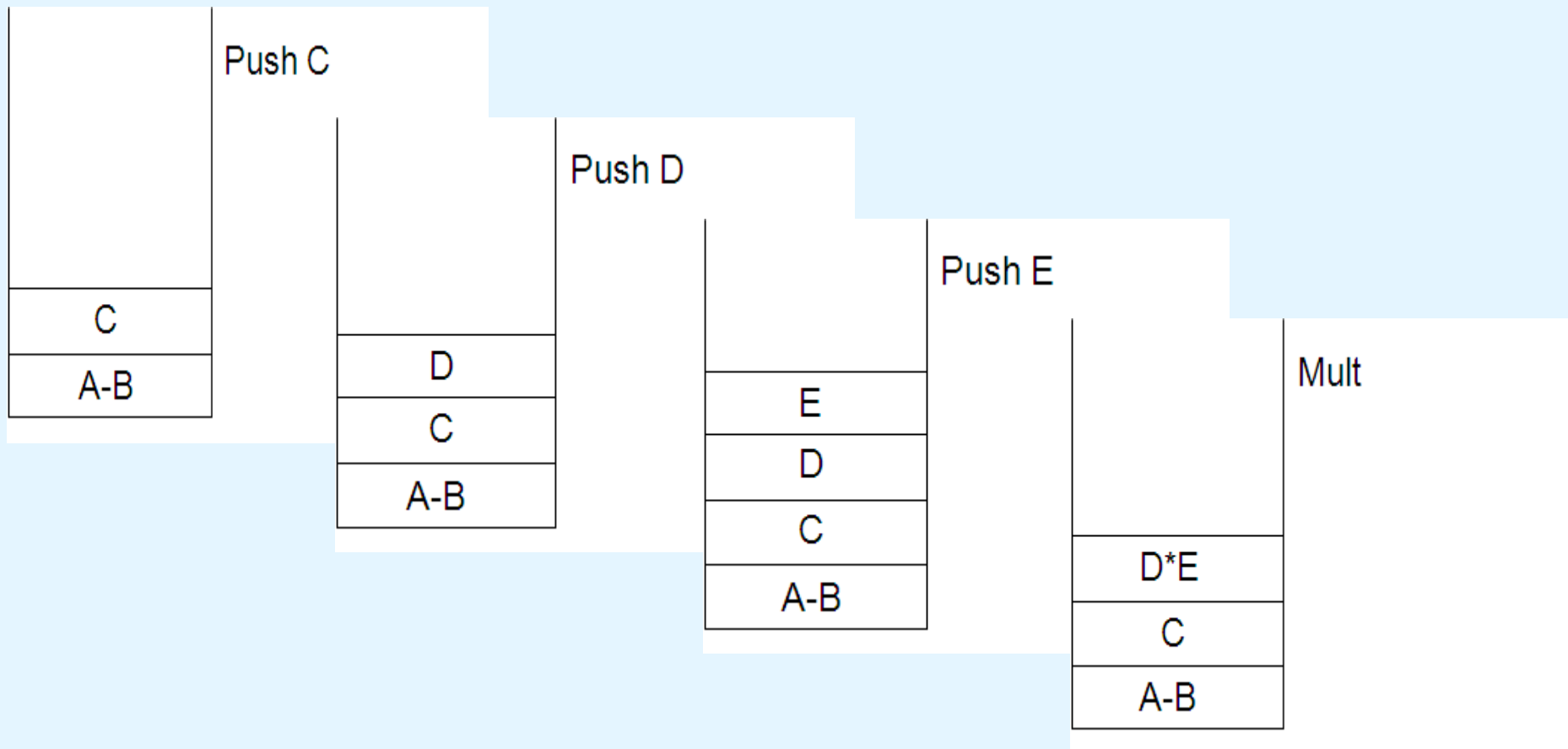
A B – C D E * F - * + G H K * + /



SBA3 b) Write an assembly program that evaluates X using a stack-based architecture.

Solution – b)

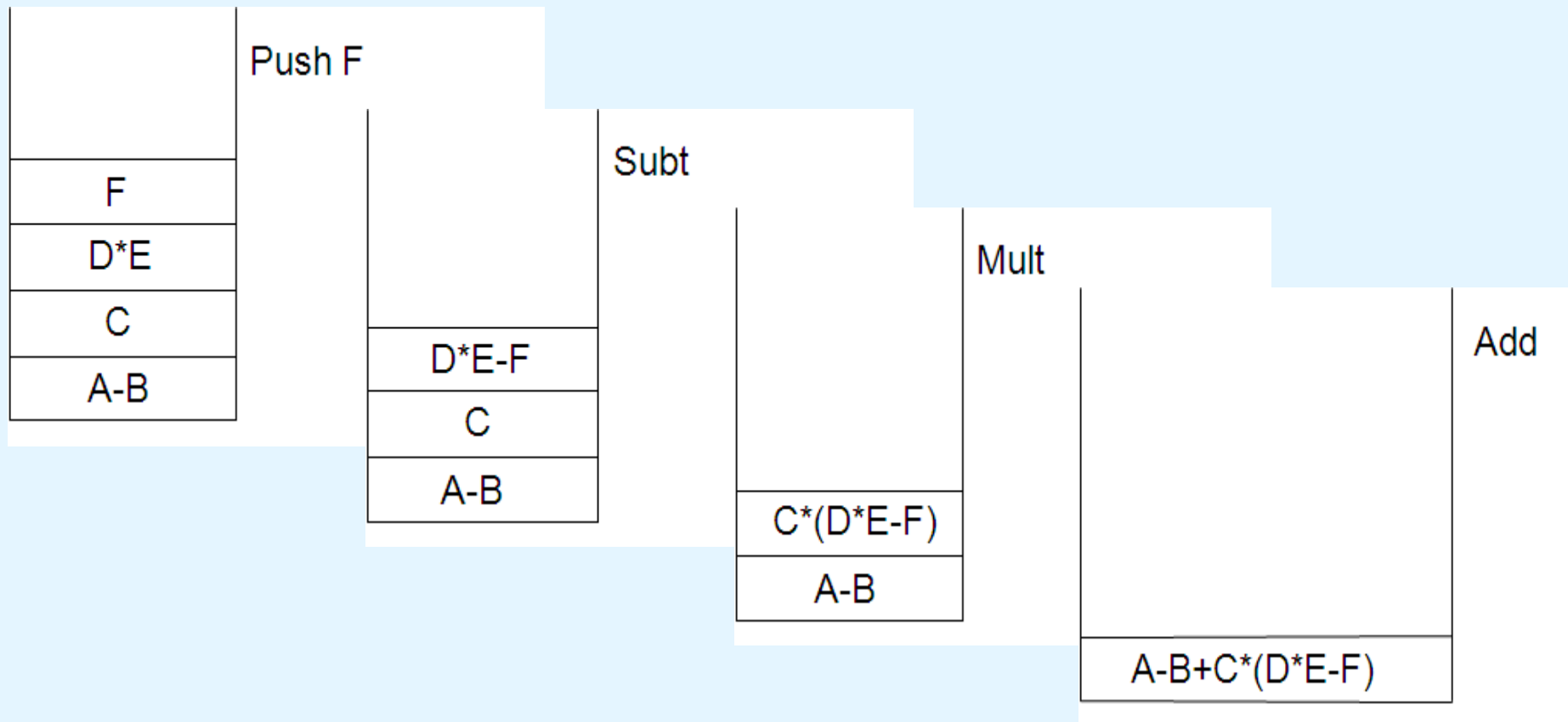
A B – C D E * F - * + G H K * + /



SBA3 b) Write an assembly program that evaluates X using a stack-based architecture.

Solution – b)

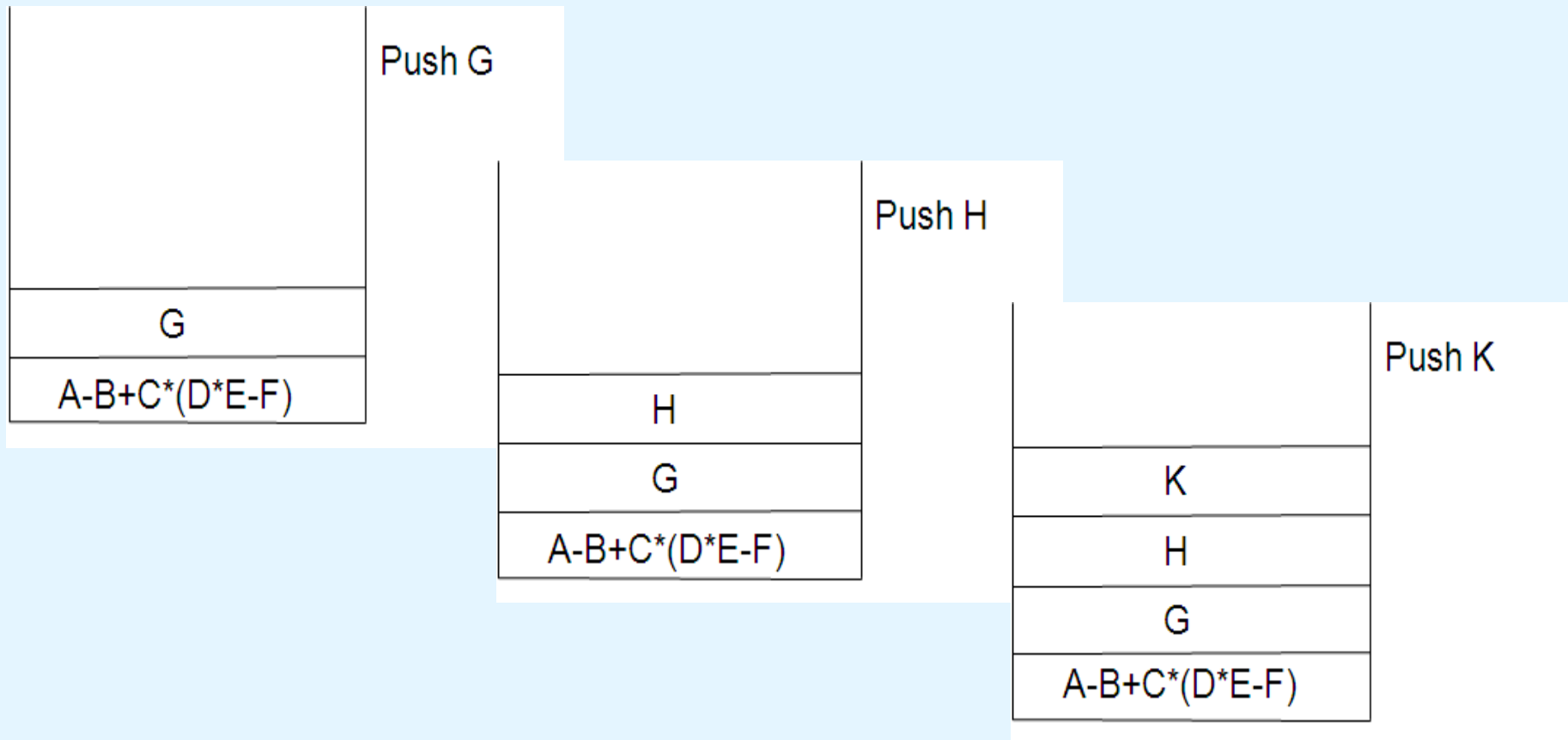
A B - C D E * F - * + G H K * + /



SBA3 b) Write an assembly program that evaluates X using a stack-based architecture.

Solution – b)

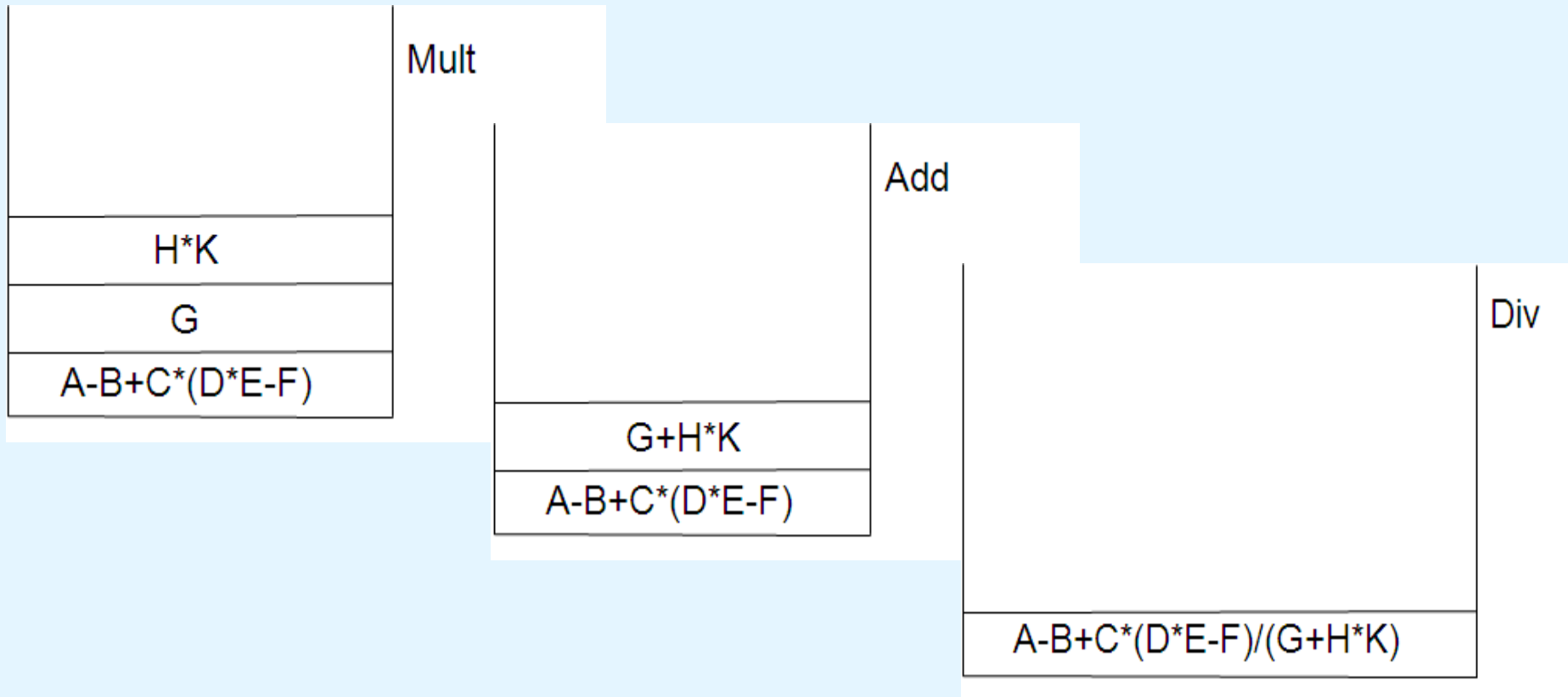
A B - C D E * F - * + G H K * + /



SBA3 b) Write an assembly program that evaluates X using a stack-based architecture.

Solution – b)

A B - C D E * F - * + G H K * + /



SBA3 b) Write an assembly program that evaluates X using a stack-based architecture.

Solution – b)

$$X = A B - C D E * F - * + G H K * + /$$

pop X

X

A-B+C*(D*E-F)/(G+H*K)

```
Push A
Push B
Subtract
Push C
Push D
Push E
Mult
Push F
Subtract
Mult
Add
Push G
Push H
Push K
Mult
Add
Div
Pop X
```

NOIS1 Write assembly code that implements $A = (B + C) * (D + E)$ on 3-, 2-, 1- and 0-address machines. Note: preserve the values of B, C, D and E.

NOIS1 Write assembly code that implements $A = (B + C) * (D + E)$ on 3-, 2-, 1- and 0-address machines. Note: preserve the values of B, C, D and E.

Solution:

Add R1, B, C
Add R2, D, E
Mult A, R1, R2

3-address machine

Load R1, B
Add R1, C
Load R2, D
Add R2, E
Mult R2, R1
Store A, R2

2-address machine

Load B
Add C
Store Temp
Load D
Add E
Mult Temp
Store A

1-address machine

Push B
Push C
Add
Push D
Push E
Add
Mult
Pop A

0-address machine

NOIS2 Remember exercise NOIS1 and derive the minimum memory space (in bytes) necessary to hold each one of the four programs, assuming that the maximum addressable RAM has 16K words.

NOIS2 Remember exercise NOIS1 and derive the minimum memory space (in bytes) necessary to hold each one of the four programs, assuming that the maximum addressable RAM has 16K words.

Solution:

common assumptions for all scenarios:

- for each scenario, the number of bits of the instructions is fixed (i.e., all instructions from the same scenario share the same number of bits, even if such implies, for some instructions, that some of its bits are left unused)

common facts for all scenarios:

- once there are no more than 16K memory words, a memory address will not need more than 14 bits (assuming the memory is word addressable):

$$\#WORDS(RAM)=16K \text{ words}$$

$$\#BITS(RAM \text{ address})=\log_2(16K) = \log_2(2^4 \cdot 2^{10})=14 \text{ bits}$$

- thus, any time an instruction contains an address, it will assign 14 bits for it

NOIS2 Remember exercise NOIS1 and derive the minimum memory space (in bytes) necessary to hold each one of the four programs, assuming that the maximum addressable RAM has 16K words.

Solution:

a) “0-address-machine” scenario

- instruction formats: |opcode|address| (*) , |opcode| (**)
- assuming the ISA only has 4 instructions (ADD, MULT, PUSH and POP), only 2 bits are necessary for the opcode:
#INST(ISA) = 4 instructions
#BITS(opcode) = $\log_2(\text{\#INST(ISA)}) = \log_2(4) = 2$ bits
- the instructions that need more bits are instructions (*):
 - 2 bits for the opcode
 - 14 bits for one memory address (A to E)
in a total of 16 bits = 2 bytes
- the number of bits per instruction is fixed => all instructions will be 2 bytes wide
- the program has **8 instructions**, and so it will consume $8 * 2 = \mathbf{16 \text{ bytes}}$

Push B	(*)
Push C	(*)
Add	(**)
Push D	(*)
Push E	(*)
Add	(**)
Mult	(**)
Pop A	(*)

0-address machine

NOIS2 Remember exercise NOIS1 and derive the minimum memory space (in bytes) necessary to hold each one of the four programs, assuming that the maximum addressable RAM has 16K words.

Solution:

b) “1-address-machine” scenario

- instruction formats: |opcode|address|
- assuming the ISA only has 4 instructions (ADD, MULT, LOAD and STORE), only 2 bits are necessary for the opcode:
#INST(ISA) = 4 instructions
 $\#BITS(opcode) = \log_2(\#INST(ISA)) = \log_2(4) = 2$ bits
- all instructions require the same number of bits:
 - 2 bits for the opcode
 - 14 bits for one memory address (A to E, Temp)
in a total of 16 bits = 2 bytes
- the number of bits per instruction is fixed => all instructions will be 2 bytes wide
- the program has **7 instructions**, and so it will consume $7 * 2 = 14$ bytes

Load B
Add C
Store Temp
Load D
Add E
Mult Temp
Store A

1-address machine

NOIS2 Remember exercise NOIS1 and derive the minimum memory space (in bytes) necessary to hold each one of the four programs, assuming that the maximum addressable RAM has 16K words.

Solution:

c) “2-address-machine” scenario

- instruction formats: $\text{opcode|register|address|}$ (*)
 $\text{opcode|register|register|}$ (**)
- assuming the ISA only has only 4 instructions (ADD, MULT, LOAD and STORE), only 2 bits are necessary for the opcode:
 $\#INST(ISA) = 4$ instructions
 $\#BITS(opcode) = \log_2(\#INST(ISA)) = \log_2(4) = 2$ bits
- assuming there are only 2 registers (R1 and R2), 1 bit is enough to identify them:
 $\#REGISTERS(ISA) = 2$ registers
 $\#BITS(register\ id) = \log_2(\#REGISTERS(ISA)) = \log_2(2) = 1$ bit

Load R1, B	(*)
Add R1, C	(*)
Load R2, E	(*)
Add R2, E	(*)
Mult R2, R1	(**)
Store A, R2	(*)

2-address machine

NOIS2 Remember exercise NOIS1 and derive the minimum memory space (in bytes) necessary to hold each one of the four programs, assuming that the maximum addressable RAM has 16K words.

Solution:

c) “3-address-machine” scenario

- instruction formats: |opcode|register|address| (*)
|opcode|register|register| (**)

Load R1, B	(*)
Add R1, C	(*)
Load R2, E	(*)
Add R2, E	(*)
Mult R2, R1	(**)
Store A, R2	(*)

2-address machine

- the instructions that needs more bits are instructions (*):
 - 2 bits for the opcode
 - 1 bit for the register id (R1 ou R2)
 - 14 bits for 1 memory address (A to E),
in a total of 17 bits that round up to the nearest multiple of 8 (24 bits = 3 bytes)
- the number of bits per instruction is fixed => all instructions will be 3 bytes wide
- the program has **6 instructions**, and so it will consume $6 * 3 = 18$ bytes

NOIS2 Remember exercise NOIS1 and derive the minimum memory space (in bytes) necessary to hold each one of the four programs, assuming that the maximum addressable RAM has 16K words.

Solution:

d) “3-address-machine” scenario

- instruction formats: $\text{opcode|register|address|address|}$ (*)
 $\text{opcode|address|register|register|}$ (**)
- assuming the ISA only has only 2 instructions (ADD and MULT), only 1 bit is necessary for the opcode:
 $\#INST(ISA) = 2 \text{ instructions}$
 $\#BITS(opcode) = \log_2(\#INST(ISA)) = \log_2(2) = 1 \text{ bit}$
- assuming there are only 2 registers (R1 and R2), 1 bit is enough to identify them:
 $\#REGISTERS(ISA) = 2 \text{ registers}$
 $\#BITS(register \text{ id}) = \log_2(\#REGISTERS(ISA)) = \log_2(2) = 1 \text{ bit}$

Add R1, B, C	(*)
Add R2, D, E	(*)
Mult A, R1, R2	(**)
3-address machine	

NOIS2 Remember exercise NOIS1 and derive the minimum memory space (in bytes) necessary to hold each one of the four programs, assuming that the maximum addressable RAM has 16K words.

Solution:

d) “3-address-machine” scenario

- instruction formats: |opcode|register|address|address| (*)
|opcode|address|register|register| (**)
- the instructions that needs more bits are instructions (*):
 - 1 bit for the opcode
 - 1 bit for the register id (R1 ou R2)
 - 14+14=28 bits for 2 memory addresses (A to E),
in a total of 30 bits that round up to the nearest multiple of 8 (32)
- the number of bits per instruction is fixed => all instructions will be 4 bytes wide
- the program has **3 instructions**, and so it will consume $3 * 4 = 12$ bytes

Add R1, B, C	(*)
Add R2, D, E	(*)
Mult A, R1, R2	(**)
3-address machine	

NOIS2 Remember exercise NOIS1 and derive the minimum memory space (in bytes) necessary to hold each one of the four programs, assuming that the maximum addressable RAM has 16K words.

Add R1, B, C
Add R2, D, E
Mult A, R1, R2

3-address machine

Load R1, B
Add R1, C
Load R2, E
Add R2, E
Mult R2, R1
Store A, R2

2-address machine

Load B
Add C
Store Temp
Load D
Add E
Mult Temp
Store A

1-address machine

Push B
Push C
Add
Push D
Push E
Add
Mult
Store A

0-address machine

Solution
(wrap up):

scenario	0-addr	1-addr	2-addr	3-addr
#instructions	8	7	6	3 (3/8=37,5%)
#bytes	16	14	18	12 (12/16=75%)

- less instructions do not necessarily imply less space
(less instructions necessary => each instruction is wider)
- the ratios of instructions and space are not necessarily the same

NOIS3 Consider the code below, suitable for running on a stack-based architecture. Assume that this code runs on a computer with a 128 x 16 bits memory, with an instruction format `|opcode|address|`, and that any instruction will occupy one word of memory.

Present the quantities requested below, in the form of a single integer number (do not present the result as a product, or using powers, or other abbreviated forms).

```
PUSH X
PUSH Y
ADD
PUSH W
PUSH Z
DIV
MUL
PUSH V
PUSH U
SUBT
DIV
NOT
POP Q
```

- a)** Provide the total number of bits (length) of the instructions.
- b)** Provide the number of bits of the "address" field of the instructions, in order to guarantee the possibility of accessing any address in the computer's memory.
- c)** Taking into account the values determined in a) and b), provide the number of bits of the "opcode" field of the instructions.
- d)** Provide the number of possible (different) opcodes supported by the given instruction format.
- e)** Provide the number of (different) opcodes actually needed to support the given program.
- f)** Provide the number of bytes occupied by the given program (including the variables used by the program)

NOIS3 Consider the code below, suitable for running on a stack-based architecture. Assume that this code runs on a computer with a 128 x 16 bits memory, with an instruction format [opcode|address|, and that any instruction will occupy one word of memory.

a) Provide the total number of bits (length) of the instructions.

```
PUSH X  
PUSH Y  
ADD  
PUSH W  
PUSH Z  
DIV  
MUL  
PUSH V  
PUSH U  
SUBT  
DIV  
NOT  
POP Q
```

Solution: 16 bits

Justification: if “any instruction will occupy one word of memory” and this computer having a 128 x 16 bits memory, it is reasonable to assume that the word size will be 16 bits and therefore that will also be the number of bits in the instructions.

NOIS3 Consider the code below, suitable for running on a stack-based architecture. Assume that this code runs on a computer with a 128 x 16 bits memory, with an instruction format |opcode| address|, and that any instruction will occupy one word of memory.

```
PUSH X
PUSH Y
ADD
PUSH W
PUSH Z
DIV
MUL
PUSH V
PUSH U
SUBT
DIV
NOT
POP Q
```

b) Provide the number of bits of the "address" field of the instructions, in order to guarantee the possibility of accessing any address in the computer's memory.

Solution: 7 bits

Justification: having this computer a 128 x 16 bits memory, and assuming 16 bits word addressing, then the memory will have 128 words, with addresses 0 to 127; these addresses, in binary, are written with $\log_2(128) = 7$ bits.

NOIS3 Consider the code below, suitable for running on a stack-based architecture. Assume that this code runs on a computer with a 128 x 16 bits memory, with an instruction format |opcode| address|, and that any instruction will occupy one word of memory.

c) Taking into account the values determined in a) and b), provide the number of bits of the "opcode" field of the instructions.

Solution: 9 bits

Justification: this computer has an instruction format |opcode| address| ; we already know, from a), that an instruction occupies one word of memory, that is, 16 bits; and we also know, from b), that in an instruction, the field |address| occupies 7 bits; thus, the field |opcode| will occupy $16 - 7 = 9$ bits.

```
PUSH X
PUSH Y
ADD
PUSH W
PUSH Z
DIV
MUL
PUSH V
PUSH U
SUBT
DIV
NOT
POP Q
```

NOIS3 Consider the code below, suitable for running on a stack-based architecture. Assume that this code runs on a computer with a 128 x 16 bits memory, with an instruction format |opcode| address|, and that any instruction will occupy one word of memory.

d) Provide the number of possible (different) opcodes supported by the given instruction format.

```
PUSH X
PUSH Y
ADD
PUSH W
PUSH Z
DIV
MUL
PUSH V
PUSH U
SUBT
DIV
NOT
POP Q
```

Solution: **512** opcodes

Justification: based on c), we know that the field |opcode| has 9 bits; hence, it would be possible to accommodate in that field $2^9 = 512$ different combinations of 9 bits, that is, 512 different opcodes (although is true that in this program we only use 7 different instructions, that is, we use only 7 of the 512 that would be possible).

NOIS3 Consider the code below, suitable for running on a stack-based architecture. Assume that this code runs on a computer with a 128 x 16 bits memory, with an instruction format `|opcode|address|`, and that any instruction will occupy one word of memory.

```
PUSH X
PUSH Y
ADD
PUSH W
PUSH Z
DIV
MUL
PUSH V
PUSH U
SUBT
DIV
NOT
POP Q
```

e) Provide the number of (different) opcodes actually needed to support the given program.

Solution: **7** opcodes

Justification: the provided program has 7 different instructions (PUSH, SUBT, DIV, ADD, MUL, NOT and POP), and so if we only wrote programs with those 7 instructions, we would only need 7 different opcodes (one per different instruction).

NOIS3 Consider the code below, suitable for running on a stack-based architecture. Assume that this code runs on a computer with a 128 x 16 bits memory, with an instruction format [opcode|address|, and that any instruction will occupy one word of memory.

f) Provide the number of bytes occupied by the given program (including the variables used by the program).

```
PUSH X  
PUSH Y  
ADD  
PUSH W  
PUSH Z  
DIV  
MUL  
PUSH V  
PUSH U  
SUBT  
DIV  
NOT  
POP Q
```

Solution: **26+14=40** bytes

Justification: based on the problem statement, we know that “any instruction will occupy one word of memory” and we already assumed that each word corresponds to 16 bits, that is, 2 bytes; once the program has a total of 13 instructions, they will occupy $13 * 2 = 26$ bytes; in addition, the program uses 7 different variables, so it is reasonable to assume that each variable also occupies one memory word, meaning that the 7 variables use $7 \times 2 = 14$ bytes.

NOIS4 A computer has a main memory unit with 24 bits per word. The instruction set consists of 150 different operations. All instructions have an operation code part (opcode) and an address part (allowing for only one address). Each instruction is stored in one word of memory.

- a) How many bits are needed for the opcode ?**
- b) How many bits are left for the address part of the instruction?**
- c) What is the maximum allowable size (in bytes) for memory?**
- d) What is the largest representable unsigned binary number ?**

NOIS4 A computer has a main memory unit with 24 bits per word. The instruction set consists of 150 different operations. All instructions have an operation code part (opcode) and an address part (allowing for only one address). Each instruction is stored in one word of memory.

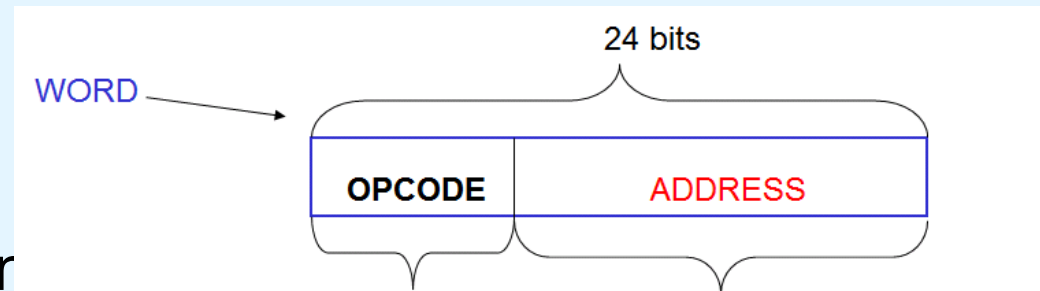
- a) How many bits are needed for the opcode ?
- b) How many bits are left for the address part of the instruction?
- c) What is the maximum allowable size (in bytes) for memory?
- d) What is the largest representable unsigned binary number ?

Problem Data:

#INST(ISA) = 150 instruction

instruction = <opcode, address>

#BITS(INST) = #BITS(word) = 24 bits



NOIS4 A computer has a main memory unit with 24 bits per word. The instruction set consists of 150 different operations. All instructions have an operation code part (opcode) and an address part (allowing for only one address). Each instruction is stored in one word of memory.

a) How many bits are needed for the opcode?

Solution:

$$\#BITS(\text{opcode}) \geq \log_2 (\#INST(ISA)) = \log_2(150) = 7, \dots$$

therefore,

$$\#BITS_{\min}(\text{opcode}) = \text{ceil}(7, \dots) = \mathbf{8 \text{ bits}}$$

NOIS4 A computer has a main memory unit with 24 bits per word. The instruction set consists of 150 different operations. All instructions have an operation code part (opcode) and an address part (allowing for only one address). Each instruction is stored in one word of memory.

b) How many bits are left for the address part of the instruction?

Solution:

$\#BITS(instruction) = \#BITS(opcode) + \#BITS(address)$

therefore,

$$\begin{aligned}\#BITS(address) &= \#BITS(instruction) - \#BITS(opcode) \\ &= 24 - 8 \\ &= \mathbf{16 \text{ bits}}\end{aligned}$$

NOIS4 A computer has a main memory unit with 24 bits per word. The instruction set consists of 150 different operations. All instructions have an operation code part (opcode) and an address part (allowing for only one address). Each instruction is stored in one word of memory.

c) What is the maximum allowable size (in bytes) for memory?

Solution:

$$\begin{aligned}\#WORDS_{\max}(\text{RAM}) &= 2^{\#BITS(\text{address})} \\ &= 2^{16} = 64\text{K words}\end{aligned}$$

$$\#BYTES(\text{word}) = \#BITS(\text{word}) / 8 \text{ bits} = 24 / 8 = 3 \text{ bytes}$$

$$\begin{aligned}\#BYTES_{\max}(\text{RAM}) &= \#WORDS_{\max}(\text{RAM}) \times \#BYTES(\text{word}) \\ &= 64\text{K} \times 3 \\ &= \mathbf{192 \text{ Kbytes}}\end{aligned}$$

NOIS4 A computer has a main memory unit with 24 bits per word. The instruction set consists of 150 different operations. All instructions have an operation code part (opcode) and an address part (allowing for only one address). Each instruction is stored in one word of memory.

d) What is the largest representable unsigned binary number ?

Solution:

#BITS(word)=24 bits

therefore,

the largest unsigned representable number is $2^{24} - 1$

NOIS5 The memory unit of a computer has 256K words of 32 bits each. Each instruction occupies a full memory word and has 4 fields: the opcode; a mode, to specify 1 of 7 addressing modes; a register address, to specify 1 of 60 registers; and a memory address.

- a) How many bits are necessary for the mode field ?**
 - b) How many bits are necessary for the register field ?**
 - c) How many bits are necessary for the address field ?
(assuming the memory is word-addressable)**
 - d) How many bits are left for the opcode ?**
- And how many different instructions are possible ?**

NOIS5 The memory unit of a computer has 256K words of 32 bits each. Each instruction occupies a full memory word and has 4 fields: the opcode; a mode, to specify 1 of 7 addressing modes; a register address, to specify 1 of 60 registers; and a memory address.

- a) How many bits are necessary for the mode field ?
 - b) How many bits are necessary for the register field ?
 - c) How many bits are necessary for the address field ?
(assuming the memory is word-addressable)
 - d) How many bits are left for the opcode ?
- And how many different instructions are possible ?

Problem Data:

#WORDS(RAM)=**256K** words

#BITS(instruction) = #BITS(RAM word)= 32 bits

instruction=<opcode,

address mode (1-**7**),

register address (1-**60**),

RAM address>

NOIS5 The memory unit of a computer has 256K words of 32 bits each. Each instruction occupies a full memory word and has 4 fields: the opcode; a mode, to specify 1 of 7 addressing modes; a register address, to specify 1 of 60 registers; and a memory address.

a) How many bits are necessary for the mode field ?

Solution:

$$\#BITS(\text{address mode}) \geq \log_2(7) = 2, \dots$$

therefore,

$$\#BITS_{\min}(\text{address mode}) = \text{ceil}(2, \dots) = \mathbf{3 \text{ bits}}$$

NOIS5 The memory unit of a computer has 256K words of 32 bits each. Each instruction occupies a full memory word and has 4 fields: the opcode; a mode, to specify 1 of 7 addressing modes; a register address, to specify 1 of 60 registers; and a memory address.

b) How many bits are necessary for the register field ?

Solution:

$$\#BITS(\text{register address}) \geq \log_2(60) = 5, \dots$$

therefore,

$$\#BITS_{\min}(\text{register address}) = \text{ceil}(5, \dots) = \mathbf{6 \text{ bits}}$$

NOIS5 The memory unit of a computer has 256K words of 32 bits each. Each instruction occupies a full memory word and has 4 fields: the opcode; a mode, to specify 1 of 7 addressing modes; a register address, to specify 1 of 60 registers; and a memory address.

**c) How many bits are necessary for the address field ?
(assuming the memory is word-addressable)**

Solution:

$$\begin{aligned}\text{\#BITS(RAM address)} &\geq \log_2(\mathbf{256K}) = \log_2(2^8 \times 2^{10}) \\ &= 18 \text{ bits}\end{aligned}$$

therefore,

$$\text{\#BITS}_{\min}(\text{RAM address}) = \text{ceil}(18) = \mathbf{18 \text{ bits}}$$

NOIS5 The memory unit of a computer has 256K words of 32 bits each. Each instruction occupies a full memory word and has 4 fields: the opcode; a mode, to specify 1 of 7 addressing modes; a register address, to specify 1 of 60 registers; and a memory address.

- d) How many bits are left for the opcode ?**
And how many different instructions are possible ?

Solution:

$$\begin{aligned}\text{\#BITS(instruction)} &= \text{\#BITS(opcode)} + \text{\#BITS(address mode)} + \\ &\quad \text{\#BITS(register address)} + \text{\#BITS(RAM address)} \\ &= \text{\#BITS(opcode)} + 3 + 6 + 18 = 32 \text{ bits}\end{aligned}$$

therefore,

$$\text{\#BITS}_{(\text{opcode})} = 32 - (3 + 6 + 18) = 32 - 27 = \mathbf{5 \text{ bits}}$$

(and, as such, $\text{\#INST}_{\text{max}}(\text{ISA}) = 2^5 = 32$ instructions)

EO1 A computer has 32-bit instructions, 12-bit memory addresses, and already supports 250 2-address instructions.
a) How many 1-address instructions can still be defined?

Note: also present the corresponding decoding algorithm.

EO1 A computer has 32-bit instructions, 12-bit memory addresses, and already supports 250 2-address instructions.
a) How many 1-address instructions can still be defined?

Solution:

00000000	xxxxxxxxxxxx	xxxxxxxxxxxx	}	250 2-address instructions
11111001	xxxxxxxxxxxx	xxxxxxxxxxxx		
11111010	000000000000	xxxxxxxxxxxx	}	2^{12}
11111010	111111111111	xxxxxxxxxxxx		
...			}	$6 * 2^{12}$ 1-address instructions
11111111	000000000000	xxxxxxxxxxxx		
11111111	111111111111	xxxxxxxxxxxx		

Decoding algorithm:

```
if (leftmost 8 bits < 11111010) {  
    execute 2-address instruction  
}  
else {  
    execute 1-address instruction  
}
```

EO1 A computer has 32-bit instructions, 12-bit memory addresses, and already supports 250 2-address instructions.
b) How many 0-address instructions can still be defined?

Note: also present the corresponding decoding algorithm.

EO1 A computer has 32-bit instructions, 12-bit memory addresses, and already supports 250 2-address instructions.
b) How many 0-address instructions can still be defined?

Solution:

Considering the solution of a), if the last 12 bits are also available for the opcode, then after the first 8 bits (which can only vary 6 times), there are 24 bits that can vary completely (2^{24} times). Therefore, the total number of instructions possible with 0 addresses is: 6×2^{24} instructions

Decoding algorithm:

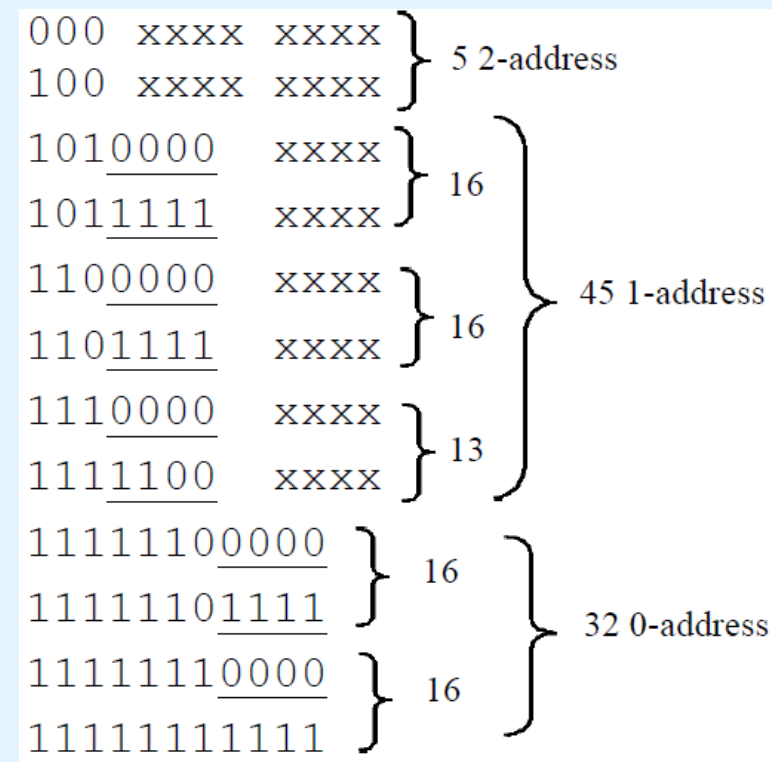
```
if (leftmost 8 bits < 11111010) {  
    execute 2-address instruction  
}  
else {  
    execute 0-address instruction  
}
```


EO2 In a computer instruction format, the instruction length is 11 bits and an address field consumes 4 bits. a) Is it possible to have 5 2-address instructions, 45 1-address instructions and 32 0-address instructions, using such format?

Note: also present the corresponding decoding algorithm.

EO2 In a computer instruction format, the instruction length is 11 bits and an address field consumes 4 bits. a) Is it possible to have 5 2-address instructions, 45 1-address instructions and 32 0-address instructions, using such format?

Solution:



Decoding algorithm:

```

if (leftmost 3 bits < 101) {
    execute 2-address instruction
}
else if (leftmost 6 bits < 111111) {
    execute 1-address instruction
}
else {
    execute 0-address instruction
}
    
```

EO2 b) Assume that a computer architect has already designed 6 two-address and 24 zero-address instructions using the instruction format defined in EO2.a). What is the maximum number of one-address instructions that can be added to the instruction set?

Note: also present the corresponding decoding algorithm.

EO2 b) Assume that a computer architect has already designed 6 two-address and 24 zero-address instructions using the instruction format defined in EO2.a). What is the maximum number of one-address instructions that can be added to the instruction set?

Solution:

Decoding algorithm:

```

if (leftmost 3 bits < 110) {
    execute 2-address instruction
}
else if (leftmost 6 bits < 111111) {
    execute 1-address instruction
}
else {
    execute 0-address instruction
}
    
```

<u>000</u>	xxxx	xxxx	}	6 2-addresses
...				
<u>101</u>	xxxx	xxxx		

110	<u>0000</u>	xxxx	}	16 1-addresses
	...			
110	<u>1111</u>	xxxx		

111	<u>0000</u>	xxxx	}	14 1-addresses
	...			
111	<u>1101</u>	xxxx		

111	1110	<u>1000</u>	}	24 0-addresses
		...		
111	1110	<u>1111</u>		
111	1111	<u>0000</u>		
		...		
111	1111	<u>1111</u>		

EO3 A computer has 24-bit instructions and 8-bit (RAM) addresses. With 34 instructions of 2 addresses already defined, how many instructions could still be accommodated with enough room for a) 1 address, b) 0 addresses? (consider a) and b) independently)

EO3 A computer has 24-bit instructions and 8-bit (RAM) addresses. With 34 instructions of 2 addresses already defined, how many instructions could still be accommodated with enough room for a) 1 address, b) 0 addresses? (consider a) and b) independently)

Solution:

With instructions of 24 bits and addresses of 8 bits, an instruction of 2 addresses has the format | opcode (8 bits) | address1 (8 bits) | address2 (8 bits) | , where $8 = 24 - (8 + 8)$.

With 34 instructions of 2 addresses already defined, and assuming a sequential use of the bit combinations available for the opcode, the combinations 0 to 33 where used for the instructions of 2 addresses, that is:

opcode (8 bits)	address1 (8 bits)	address2 (8 bits)	
0 = 00000000	XXXXXXXX	XXXXXXXX	
...			
33 = 00100001	XXXXXXXX	XXXXXXXX	

EO3 A computer has 24-bit instructions and 8-bit (RAM) addresses. With 34 instructions of 2 addresses already defined, how many instructions could still be accommodated with enough room for a) 1 address, b) 0 addresses? (consider a) and b) independently)

Solution:

With instructions of 24 bits and addresses of 8 bits, an instruction of 2 addresses has the format | opcode (8 bits) | address1 (8 bits) | address2 (8 bits) | , where $8 = 24 - (8 + 8)$.

With 34 instructions of 2 addresses already defined, and assuming a sequential use of the bit combinations available for the opcode, the combinations 0 to 33 where used for the instructions of 2 addresses, that is:

opcode (8 bits)	address1 (8 bits)	address2 (8 bits)	
0 = 00000000	XXXXXXXX	XXXXXXXX	
...			
33 = 00100001	XXXXXXXX	XXXXXXXX	

a) An instruction of 1 address will have the format | opcode (8+8 bits) | address (8 bits) |. However, the first 8 bits (prefix) of the opcode can only vary from 34 to 255, while the second 8 bits (suffix) can vary fully (from 0 to $2^8-1=255$). Thus, for each prefix between 34 and 255, the opcode can have $2^8=256$ suffixes (from 0 to 255), which gives a total of $(255-34+1) * 256 = 56832$ instructions of 1 address.

opcode prefix (8 bits)	opcode suffix (8 bits)	address (8 bits)	
34 = 00100010	2^8 combinations: 0 to 255	XXXXXXXX	
...			
255 = 11111111	2^8 combinations: 0 to 255	XXXXXXXX	

EO3 A computer has 24-bit instructions and 8-bit (RAM) addresses. With 34 instructions of 2 addresses already defined, how many instructions could still be accommodated with enough room for a) 1 address, b) 0 addresses? (consider a) and b) independently)

Solution:

With instructions of 24 bits and addresses of 8 bits, an instruction of 2 addresses has the format | opcode (8 bits) | address1 (8 bits) | address2 (8 bits) | , where $8 = 24 - (8 + 8)$.

With 34 instructions of 2 addresses already defined, and assuming a sequential use of the bit combinations available for the opcode, the combinations 0 to 33 where used for the instructions of 2 addresses, that is:

opcode (8 bits)	address1 (8 bits)	address2 (8 bits)	
0 = 00000000	XXXXXXXX	XXXXXXXX	
...			
33 = 00100001	XXXXXXXX	XXXXXXXX	

b) An instruction of 0 addresses will have the format | opcode (8+16 bits) |, in which the 8 bits of the opcode prefix can only vary from 34 to 255, while the 16 bits of the suffix can vary fully (from 0 to $2^{16}-1=65535$). Thus, for each prefix between 34 and 255, the opcode can have $2^{16}=65536$ suffixes (from 0 to 65535), which gives a total of $(255-34+1) * 65536 = 14548992$ instructions of 0 addresses.

opcode prefix (8 bits)	opcode suffix (16 bits)	
34 = 00100010	2^{16} combinations: 0 to 65535	
...		
255 = 11111111	2^{16} combinations: 0 a 65535	

EO4 A computer has 32-bit instructions and 8-bit (RAM) addresses. Answer the following questions:

- a)** Show how you would consume the available combinations of 32 bits to simultaneously accommodate 64 instructions with 3 addresses and 256 instructions with 2 addresses.
- b)** Using as a starting point the result of a), how many instructions with 1 address would it be possible to accommodate? (Justify by showing how the 32-bit combinations remaining from a) are consumed).
- c)** Using as a starting point the result of a), how many instructions with 0 addresses would it be possible to accommodate? (Justify by showing how the 32-bit combinations remaining from a) are consumed).

NOTE:

consume the bit combinations in ascending order (from 32 0's to 32 1's).

EO4 A computer has 32-bit instructions and 8-bit (RAM) addresses. Answer the following questions:

a) Show how you would consume the available combinations of 32 bits to simultaneously accommodate 64 instructions with 3 addresses and 256 instructions with 2 addresses.

Solution:

With instructions of 32 bits and addresses of 8 bits, an instruction with 3 addresses has the format | opcode (8 bits) | address1 (8 bits) | address2 (8 bits) | address3 (8 bits) |, where $8 = 32 - (8 + 8 + 8)$. Assuming a sequential use of the bit combinations available for the opcode, the combinations 0 to 63 ensure 64 instructions of 3 addresses, that is:

opcode (8 bits)	address1 (8 bits)	address2 (8 bits)	address3 (8 bits)	
0 = 00000000	XXXXXXXX	XXXXXXXX	XXXXXXXX	
...				
63 = 00111111	XXXXXXXX	XXXXXXXX	XXXXXXXX	

An instruction of 2 addresses has the format | opcode (8+8 bits) | address1 (8 bits) | address2 (8 bits) |. Continuing to consume the bit combinations still available, the 8 bits of opcode prefix can still vary from 64 to 255, and the 8 bits of the suffix can fully vary (from 0 to $2^8-1=255$). Thus, it is possible to accommodate $(255-64+1) * 2^8 = 49152$ instructions of 2 addresses, much more than the 256 asked !

opcode prefix (8 bits)	opcode suffix (8 bits)	address1 (8 bits)	address2 (8 bits)	
64 = 01000000	2^8 combinations: 0 to 255	XXXXXXXX	XXXXXXXX	
...				
255 = 11111111	2^8 combinations: 0 to 255	XXXXXXXX	XXXXXXXX	

EO4 A computer has 32-bit instructions and 8-bit (RAM) addresses. Answer the following questions:

b) Using as a starting point the result of a), how many instructions with 1 address would it be possible to accommodate? (Justify by showing how the 32-bit combinations remaining from a) are consumed).

Solution:

Starting point (64 instructions of 3 addresses, and 256 instructions of 2 addresses):

opcode (8 bits)	address1 (8 bits)	address2 (8 bits)	address3 (8 bits)
0 = 00000000	XXXXXXXX	XXXXXXXX	XXXXXXXX
...			
63 = 00111111	XXXXXXXX	XXXXXXXX	XXXXXXXX

opcode prefix (8 bits)	opcode suffix (8 bits)	address1 (8 bits)	address2 (8 bits)
64 = 01000000	2^8 combinations: 0 to 255	XXXXXXXX	XXXXXXXX

An instruction of 1 address has the format | opcode ($8+16$ bits) | address (8 bits) |. The opcode prefix of 8 bits can vary from 65 to 255, and the suffix of 16 bits can fully vary (from 0 to $2^{16}-1=65535$):

opcode prefix (8 bits)	opcode suffix (16 bits)	address (8 bits)
65 = 01000001	2^{16} combinations: 0 to 65535	XXXXXXXX
...		
255 = 11111111	2^{16} combinations: 0 to 65535	XXXXXXXX

It is still possible to accommodate $(255-65+1)*2^{16} = 12517376$ instructions of 1 address.

EO4 A computer has 32-bit instructions and 8-bit (RAM) addresses. Answer the following questions:

c) Using as a starting point the result of a), how many instructions with 0 addresses would it be possible to accommodate? (Justify by showing how the 32-bit combinations remaining from a) are consumed).

Solution:

Starting point (64 instructions of 3 addresses, and 256 instructions of 2 addresses):

opcode (8 bits)	address1 (8 bits)	address2 (8 bits)	address3 (8 bits)
0 = 00000000	XXXXXXXX	XXXXXXXX	XXXXXXXX
...			
63 = 00111111	XXXXXXXX	XXXXXXXX	XXXXXXXX
opcode prefix (8 bits)	opcode suffix (8 bits)	address1 (8 bits)	address2 (8 bits)
64 = 01000000	<u>2⁸</u> combinations: 0 to 255	XXXXXXXX	XXXXXXXX

An instruction of 0 addresses has the format | opcode (8+24 bits) |. The opcode prefix of 8 bits can vary from 65 to 255, and the suffix of 24 bits can fully vary (from 0 to 2²⁴-1=16777215):

opcode prefix (8 bits)	opcode suffix (24 bits)	
65 = 01000001	<u>2²⁴</u> combinations: 0 to 1677215	
...		
255 = 11111111	<u>2¹⁶</u> combinations: 0 to 1677215	

It is still possible to accommodate $(255-65+1)*2^{24} = 3204448256$ instructions of 0 addresses.



AM1 Consider the instruction Load 1000. Given main memory and register R1 contents as shown below, and assuming R1 is implied in the indexed addressing mode, determine the actual value loaded into the accumulator and complete the right table:

Memory		
1000	1400	R1 200
...		
1100	400	
...		
1200	1000	
...		
1300	1100	
...		
1400	1300	

Mode	Value Loaded into AC
Immediate	
Direct	
Indirect	
Indexed	

AM1 Consider the instruction Load 1000. Given main memory and register R1 contents as shown below, and assuming R1 is implied in the indexed addressing mode, determine the actual value loaded into the accumulator and complete the right table:

Memory

1000	1400
...	
1100	400
...	
1200	1000
...	
1300	1100
...	
1400	1300

R1

200

Solution:

Mode	Value
Immediate	1000
Direct	1400
Indirect	1300
Indexed	1000

AM2 Consider the instruction Load 500. Given main memory and register R1 contents as shown below, and assuming R1 is implied in the indexed addressing mode, determine the actual value loaded into the accumulator and complete the right table:

100	600
...	
400	300
...	
500	100
...	
600	500
...	
700	800

R1

200

Mode	Value Loaded into AC
Immediate	
Direct	
Indirect	
Indexed	

AM2 Consider the instruction Load 500. Given main memory and register R1 contents as shown below, and assuming R1 is implied in the indexed addressing mode, determine the actual value loaded into the accumulator and complete the right table:

100	600
...	
400	300
...	
500	100
...	
600	500
...	
700	800

R1

200

Solution:

Mode	Value
Immediate	500
Direct	100
Indirect	600
Indexed	800

AM3 Suppose a CPU of an Accumulator (AC) based architecture is able to use multiple modes of addressing and that the content of main memory in a certain instant is represented by the figure bellow.

Consider the execution of an instruction LOAD 1200, and assume the existence of a register R1200 with content 1800 used in addressing modes that require an auxiliary register. Indicate the numerical value that will be transferred to the AC register (accumulator) when the addressing mode used is:

- a) Immediate
- b) Direct
- c) Register Direct (using R1200)
- d) Indirect
- e) Register Indirect (using R1200)
- f) Indexed (using R1200)
- g) Based (using R1200)

address:	content:
1200	1400
1400	1600
1600	1800
1800	2000
2000	1200
3000	1200

AM3 Suppose a CPU of an Accumulator (AC) based architecture is able to use multiple modes of addressing and that the content of main memory in a certain instant is represented by the figure bellow.

Solution:

Consider the execution of an instruction LOAD 1200, and assume the existence of a register R1200 with content 1800 used in addressing modes that require an auxiliary register. Indicate the numerical value that will be transferred to the AC register (accumulator) when the addressing mode used is:

a) Immediate: $AC = 1200$

b) Direct: $AC = M[1200] = 1400$

c) Register Direct (using R1200) : $AC = R1200 = 1800$

d) Indirect: $AC = M[M[1200]] = M[1400] = 1600$

e) Register Indirect (using R1200): $AC = M[R1200] = M[1800] = 2000$

f) Indexed (using R1200): $AC = M[\text{Base} + \text{Offset}]$
 $= M[1200 + R1200] = M[1200 + 1800]$
 $= M[3000] = 1200$

g) Based (using R1200): $AC = M[\text{Base} + \text{Offset}]$
 $= M[R1200 + 1200] = M[1800 + 1200]$
 $= M[3000] = 1200$

address:	content:
1200	1400
1400	1600
1600	1800
1800	2000
2000	1200
3000	1200

ILP1 A certain CPU has a 5-stage pipeline with a 40ns clock cycle. Without resorting to the pipeline, the processing of an instruction on this CPU takes, on average, 200ns. Determine (*): a) the speedup offered by the pipeline for the execution of a program with 400 instructions; b) the maximum theoretical speedup providable by the pipeline.

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

with

n = num. of instructions (tasks)
t_i = duration of an instruction (task)
k = num. of pipeline stages
t_s = duration of a stage

(*) Answers truncated to the thousandths (1/1000)

ILP1 A certain CPU has a 5-stage pipeline with a 40ns clock cycle. Without resorting to the pipeline, the processing of an instruction on this CPU takes, on average, 200ns. Determine (*): a) the speedup offered by the pipeline for the execution of a program with 400 instructions; b) the maximum theoretical speedup providable by the pipeline.

Solution:

$n = 400$ instructions

$t_i = 200$ ns

$k = 5$ stages

$t_s = 40$ ns

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

with

n = num. of instructions (tasks)

t_i = duration of an instruction (task)

k = num. of pipeline stages

t_s = duration of a stage

a) pipeline speedup for the execution of 400 instructions

$$S_{400} = (400 \times 5) / (5 + 400 - 1) = 4.950 (*)$$

(*) Answers truncated to the thousandths (1/1000)

b) maximum theoretical speedup of the pipeline

$$\text{MAX}(S_n) = 5$$

ILP2 A certain CPU has a 5-stage pipeline with a 20ns clock cycle. Without resorting to the pipeline, the processing of an instruction on this CPU takes, on average, 100ns. Determine (*): a) the speedup offered by the pipeline for the execution of a program with 200 instructions; b) the maximum theoretical speedup providable by the pipeline.

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

with

n = num. of instructions (tasks)
t_i = duration of an instruction (task)
k = num. of pipeline stages
t_s = duration of a stage

(*) Answers truncated to the thousandths (1/1000)

ILP2 A certain CPU has a 5-stage pipeline with a 20ns clock cycle. Without resorting to the pipeline, the processing of an instruction on this CPU takes, on average, 100ns. Determine (*): a) the speedup offered by the pipeline for the execution of a program with 200 instructions; b) the maximum theoretical speedup providable by the pipeline.

Solution

$n = 100$ instructions

$t_i = 100$ ns

$k = 5$ stages

$t_s = 20$ ns

$$S_n = (n \times t_i) / [(k + n - 1) \times t_s] \\ = (n \times k) / (k + n - 1)$$

$$\text{MAX}(S_n) = k$$

with

n = num. of instructions (tasks)

t_i = duration of an instruction (task)

k = num. of pipeline stages

t_s = duration of a stage

a) pipeline speedup for the execution of 200 instructions

$$S_{200} = (200 \times 5) / (5 + 200 - 1) = 4.901 (*)$$

(*) Answers truncated to the thousandths (1/1000)

b) maximum theoretical speedup of the pipeline

$$\text{MAX}(S_n) = 5$$

ILP3 A CPU takes 117 ns, on average, to process an instruction following the von Neumann model, in which each instruction goes through the fetch, decode and execute phases, before the next one enters the same path. An improved version of this CPU was equipped with an instruction pipeline, with 13 stages and an duration per stage of 9 ns.

Present the quantities requested below, in the form of a single number (do not present the result as a product, with powers, or other abbreviated forms; present the values (*) with precision limited to hundredths, truncated)

- a)** The time, in ns (nano-seconds) that the original CPU (not enhanced) would take to run an X program with 4000 instructions.
- b)** The time, in ns, it would take the enhanced CPU to execute program X.
- c)** (*) The acceleration (speedup) provided by the improved CPU pipeline in the execution of program X.
- d)** (*) The Efficiency with which the improved CPU pipeline is used during the execution of program X (i.e., the percentage that the value obtained in c) represents in relation to the maximum theoretical acceleration provided by the pipeline).

ILP3 A CPU takes 117 ns, on average, to process an instruction following the von Neumann model, in which each instruction goes through the fetch, decode and execute phases, before the next one enters the same path. An improved version of this CPU was equipped with an instruction pipeline, with 13 stages and an duration per stage of 9 ns.

Present the quantities requested below, in the form of a single number (do not present the result as a product, with powers, or other abbreviated forms; present the values (*) with precision limited to hundredths, truncated)

Solution:

a) The time, in ns (nano-seconds) that the original CPU (not enhanced) would take to run an X program with 4000 instructions.

$$T_{\text{non-pipeline}} = n * t_i = 4000 * 117 = 468000 \text{ ns}$$

b) The time, in ns, it would take the enhanced CPU to execute program X.

$$T_{\text{pipeline}} = (k + n - 1) * t_s = (13 + 4000 - 1) * 9 = 36108 \text{ ns}$$

ILP3 A CPU takes 117 ns, on average, to process an instruction following the von Neumann model, in which each instruction goes through the fetch, decode and execute phases, before the next one enters the same path. An improved version of this CPU was equipped with an instruction pipeline, with 13 stages and an duration per stage of 9 ns.

Present the quantities requested below, in the form of a single number (do not present the result as a product, with powers, or other abbreviated forms; present the values (*) with precision limited to hundredths, truncated)

Solution:

c) (*) The acceleration (speedup) provided by the improved CPU pipeline in the execution of program X.

$$\text{Speedup}_X = T_{\text{non-pipelined}} / T_{\text{pipelined}} = 468000 / 36108 = 12,96$$

d) (*) The Efficiency with which the improved CPU pipeline is used during the execution of program X (i.e., the percentage that the value obtained in c) represents in relation to the maximum theoretical acceleration provided by the pipeline).

$$\text{Efficiency}_X = \text{Speedup}_X / k = 12,96 / 13 = 99,69\%$$